



ESCUELA DE INGENIERÍA DE FUENLABRADA

Ingeniera en Sistemas Audiovisuales y Multimedia

TRABAJO FIN DE GRADO

DESARROLLO DE UN ENTORNO DE APRENDIZAJE
INTERACTIVO CON CHATBOT PARA LA EDUCACIÓN
INFANTIL

Autor : María Cecilia Campos Torre

Tutor : Jesús García Parrado Alameda

Curso académico 2024/2025

©2025 María Cecilia Campos Torre

Algunos derechos reservados

Este documento se distribuye bajo la licencia

“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,

disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

*Dedicado a
mi familia, mis primos, mis tíos y mis abuelos.
En especial, a mis padres, por su amor y apoyo incondicional.*

Agradecimientos

En este punto de la carrera, siendo ya el quinto año y quedándome únicamente el Trabajo de Fin de Grado, solo me queda sentirme profundamente agradecida. Agradecida por las horas que dediqué, por los días enteros que tuve que sacrificar, e incluso por aquellas asignaturas que me demostraron que estudiar puede llegar a ser encantador. Agradecida por aquellos profesores que compartieron con pasión su amplio conocimiento, y sobre todo, por los compañeros que estuvieron a mi lado durante todos estos años.

También quisiera agradecer especialmente a mi familia, y en particular, a mis padres, Fanny y Martín, por su apoyo incondicional en cada etapa. Y también a mi tutor, Jesús García Parrado Alameda, por su tiempo dedicado al seguimiento de este proyecto. Gracias a todos

Resumen

Este Trabajo Fin de Grado se centra en el diseño y desarrollo de un chatbot educativo, teniendo como principal objetivo ayudar a niños a practicar divisiones de forma interactiva, accesible y amigable. El chatbot está orientado a reforzar el aprendizaje matemático a través de una interfaz cercana y dinámica, adaptada a las necesidades de un público infantil, donde también se ha tenido en cuenta la dislexia y, por tanto, se ha buscado una tipografía amigable que facilite la lectura.

Junto con los ejercicios prácticos, también se ha diseñado contenido teórico, acompañado de imágenes y GIF, con el objetivo de facilitar la comprensión teórica de la división y reforzar el aprendizaje de forma visual.

El proyecto ha sido desarrollado utilizando Rasa como motor de procesamiento del lenguaje natural y React para la construcción del front-end. La comunicación entre ambos se realiza mediante peticiones HTTP al webhook de Rasa. Además, se han integrado librerías como Intro.js, para ofrecer una guía interactiva, y un sistema de logros visuales que refuerza el progreso del usuario mediante insignias que se pueden desbloquear. Además, se añadió un botón que activa y desactiva el modo oscuro, una lista donde se irán almacenando los ejercicios incorrectos, para después poder realizar una visualización gráfica de esos errores, con el fin de mejorar la comprensión del concepto de división.

Summary

This Final Degree Project focuses on the design and development of an educational chatbot, with the main goal of helping children practice division in an interactive, accessible, and user-friendly way. The chatbot is designed to reinforce mathematical learning through a dynamic and approachable interface, tailored to the needs of a young audience. Special attention has been given to accessibility by considering dyslexia, incorporating a friendly typeface that improves readability.

In addition to practical exercises, theoretical content has also been created, accompanied by explanatory images and GIFs, with the aim of facilitating the conceptual understanding of division and reinforcing learning through visual resources.

The project has been developed using Rasa as the natural language processing engine and React for building the front end. Communication between the two is handled through HTTP requests to Rasa's webhook. Furthermore, libraries such as Intro.js have been integrated to provide an interactive guide, along with a visual achievement system that rewards user progress with unlockable badges.

A dark mode toggle button has also been added, along with a feature that stores incorrectly answered exercises in a list. These errors can later be displayed graphically to support a better understanding of the concept of division.

Índice general

1. Introducción	1
1.1. contexto educativo	2
1.2. Estructura de la memoria	4
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
3. Tecnologías utilizadas	7
3.1. Tecnologías principales	7
3.1.1. Rasa	7
3.1.2. React	10
3.1.3. JavaScript	11
3.1.4. HTML5	11
3.1.5. CSS	12
3.1.6. Intro.js	12
3.1.7. Python	12
3.1.8. Json	13
3.1.9. Spacy	14
3.2. Tecnologías auxiliares	14
3.2.1. GitHub	14
3.2.2. Visual Studio Code	15
3.2.3. LaTeX	16

4. Diseño e implementación del proyecto	17
4.1. Planificación temporal	17
4.2. Sprint 1: Investigación sobre las tecnologías a utilizar	18
4.2.1. Objetivos	18
4.2.2. Implementación del desarrollo	19
4.3. Sprint 2: Primeros pasos en RASA	19
4.3.1. Objetivos	19
4.3.2. Implementación del desarrollo	19
4.4. Sprint 3: Entrenamiento del bot	20
4.4.1. Objetivos	20
4.4.2. Implementación del desarrollo	20
4.5. Sprint 4: Desarrollo de funcionalidades del bot	25
4.5.1. Objetivos	25
4.5.2. Implementación del desarrollo	26
4.6. Sprint 5: Creación de la interfaz web e Integración con Rasa	28
4.6.1. Objetivos	29
4.6.2. Implementación del desarrollo	29
4.7. Sprint 6: Generación de gráficos para explicar las divisiones	31
4.7.1. Objetivos	31
4.7.2. Implementación del desarrollo	31
4.8. Sprint 7: Implementación de una guía de usuario y sistemas de logros	32
4.8.1. Objetivos	32
4.8.2. Implementación del desarrollo	32
4.9. Arquitectura general	35
5. Experimentos y validación	37
5.1. Reconocimiento del lenguaje natural	37
5.2. Corrección de respuestas incorrectas	39
5.3. Casos de uso	39
5.3.1. Estructura de los casos de uso	40

<i>ÍNDICE GENERAL</i>	<i>XI</i>
6. Conclusiones	47
6.1. Consecución de objetivos	47
6.2. Aplicación de lo aprendido	48
6.3. Lecciones aprendidas	48
6.4. Trabajos futuros	49
A. Manual de usuario	51
A.1. Instalación	51
A.2. Guía usuario	52
Bibliografía	53

Índice de figuras

1.1. Porcentaje de niños fuera del sistema escolar primario, según región y grupo socioeconómico.	3
3.1. Arquitectura de RASA.	10
4.1. Flujo conversacional	25
4.2. Página Web con el chat integrado	30
4.3. Elección de divisiones mediante botones	30
4.4. Corrección visual.	32
4.5. Vista inicial de la página web al ingresar	34
4.6. Vista final de la página web en modo oscuro activado	34
5.1. Conversación coherente con el bot.	38
5.2. Comprobar si el bot es capaz de responder unas sumas planteadas de formas distintas.	39
5.3. Comprobación de almacenamiento y limpieza del slot de divisiones incorrectas	39
5.4. Ejemplo de interacción inicial. El usuario saluda y pregunta por la función del chatbot, quien responde explicando que puede ayudar a aprender y practicar divisiones.	40
5.5. El usuario expresa su deseo de aprender a dividir. El chatbot responde con un mensaje interactivo y opciones visuales para elegir el tipo de división.	41
5.6. Explicación visual y animada de una división de una cifra. El chatbot muestra cómo se reparten 10 caramelos entre dos niños, reforzando el concepto de división como reparto igualitario.	41

5.7. Explicación visual y textual sobre cómo resolver divisiones con decimales. El chatbot guía al usuario con instrucciones sencillas y una representación gráfica del procedimiento.	42
5.8. Explicación visual y textual sobre cómo resolver divisiones con decimales. El chatbot guía al usuario con instrucciones sencillas y una representación gráfica del procedimiento.	43
5.9. Explicación textual sobre cómo resolver divisiones con un divisor de dos cifras. El chatbot utiliza un ejemplo numérico simple para facilitar la comprensión. . .	43
5.10. Interacción en la que el usuario selecciona cómo desea practicar divisiones. El chatbot ofrece opciones visuales para elegir el tipo de ejercicio y el nivel de dificultad.	44
5.11. El usuario elige la cantidad de ejercicios a practicar. El chatbot muestra enunciados y proporciona feedback inmediato según la respuesta.	45
5.12. Corrección visual y explicada que proporciona el chatbot.	45

Capítulo 1

Introducción

Aunque el número de niños sin escolarizar ha disminuido ligeramente, millones aún siguen sin acceso a la educación. Según la UNESCO, *“cerca de 258 millones de niños, adolescentes y jóvenes de todo el mundo no estaban escolarizados en 2018, una cifra que representa aproximadamente un sexto de la población mundial en edad escolar (entre seis y 17 años). Estas cifras revelan que, durante más de una década, el avance ha sido mínimo o nulo y, lo que es más preocupante, que si no se toman medidas urgentes, 12 millones de niños nunca verán el interior de un aula”*[1]. Estas cifras reflejan una realidad preocupante que exige nuevas soluciones educativas capaces de llegar a quienes más lo necesiten.

En este sentido, el avance de la tecnología ha tenido un impacto especialmente notable en el ámbito educativo, convirtiéndose en una herramienta clave para el aprendizaje. Este progreso no solo ha hecho los métodos más accesibles, sino que también ha dado lugar a formas de enseñanza más visuales, interactivas y adaptativas, aspectos fundamentales para captar la atención de los estudiantes y respetar sus ritmos individuales.

Sin embargo, estos avances también traen consigo nuevos desafíos, como la desigualdad en el acceso a dispositivos e internet, o la exposición a contenidos inadecuados. Por ello, la seguridad digital se convierte en un requisito esencial en el diseño de cualquier recurso educativo en línea.

En este contexto, el presente Trabajo de Fin de Grado propone una solución educativa basada en un chatbot conversacional que:

- Ofrece **feedback en tiempo real** al corregir ejercicios automáticamente.

- Se adapta al **nivel de dificultad** y ritmo del usuario.
- Está disponible de forma **gratuita y accesible online**, sin necesidad de descarga o registro.
- Presenta un entorno **visual e interactivo**, atractivo para el público infantil.
- Funciona de forma segura, sin exponer al niño a contenido externo.

En comparación con otras soluciones existentes:

- **Fichas interactivas:** suelen ser estáticas, sin respuesta automática, y requieren intervención del docente o familiar.
- **Aplicaciones móviles:** muchas requieren instalación, registro o pagos, y no siempre ofrecen contenido adaptado a cada alumno.
- **Juegos educativos en línea:** suelen centrarse más en entretenimiento que en el refuerzo estructurado de contenidos escolares.

A diferencia de estas opciones, la solución propuesta combina los beneficios del aprendizaje autónomo con la orientación guiada de un asistente virtual, disponible desde cualquier navegador. El objetivo principal es reforzar el aprendizaje de la división, proporcionando un entorno educativo personalizado, lúdico y accesible para todos los niños.

1.1. contexto educativo

El uso de herramientas digitales en el ámbito educativo ha crecido notablemente en los últimos años. La incorporación de dispositivos electrónicos, plataformas virtuales y recursos interactivos ha transformado la manera en que los estudiantes acceden al conocimiento, permitiendo una enseñanza más flexible, visual y en muchos casos más atractiva para el alumnado.

Sin embargo, esta evolución ha puesto también de manifiesto importantes desigualdades. En numerosos países, ciudades o regiones rurales, muchos niños no pueden asistir a la escuela con regularidad, ya sea por barreras geográficas, económicas o por falta de infraestructura. En estos contextos, el acceso a internet, aunque sea limitado, puede representar una alternativa viable

para continuar con el aprendizaje, siempre que se disponga de recursos apropiados, gratuitos y adaptados a sus necesidades.

Como se observa en la figura 4.1, la situación en el África subsahariana es especialmente preocupante: el 22 % de los niños en edad de primaria no está escolarizado, una cifra que asciende al 40 % en los sectores más empobrecidos. Estos datos, proporcionados por UNICEF [2, 3], ponen en evidencia la magnitud del reto global y la urgencia de implementar soluciones educativas digitales capaces de alcanzar a los estudiantes más vulnerables.

Por otro lado, incluso en entornos donde sí existe acceso a la educación formal, las tecnologías digitales pueden desempeñar un papel clave como complemento del aprendizaje escolar. Estas herramientas permiten reforzar contenidos, fomentar la autonomía del alumno y adaptarse a distintos ritmos de aprendizaje. Mientras algunos estudiantes asimilan conceptos rápidamente, otros necesitan más tiempo, distintas explicaciones o enfoques más visuales. Sin embargo, muchas escuelas no cuentan con el tiempo, el personal ni los recursos necesarios para atender esta diversidad.

Ante esta realidad, se hace evidente la necesidad de desarrollar recursos educativos accesibles, inclusivos y diseñados específicamente para el público infantil. Dichas herramientas deben permitir que cada niño avance a su propio ritmo, consolidando conocimientos de manera personalizada y sin las presiones propias del aula tradicional. Además, deben estar disponibles sin barreras económicas ni requisitos técnicos complejos que limiten su uso.

Estos factores justifican la importancia de explorar soluciones tecnológicas que respondan a estas limitaciones y que permitan acercar una educación de calidad a todos los niños, independientemente de su contexto.

Regional aggregates (unit in %, based on >50% population coverage)												
			Total	Female	Male	Rural	Urban	Poorest	Second	Middle	Fourth	Richest
East Asia & Pacific	EAP		4	4	4	4	4	5	2	1	1	1
Europe & Central Asia	ECA		—	—	—	—	—	—	—	—	—	—
Eastern Europe & Central Asia		EECA	3	2	3	2	3	4	2	2	2	2
Western Europe		WE	—	—	—	—	—	—	—	—	—	—
Latin America & Caribbean	LAC		2	2	2	3	1	3	2	1	1	1
Middle East & North Africa		MENA	7	7	6	9	4	12	7	5	4	3
North America	NA		—	—	—	—	—	—	—	—	—	—
South Asia	SA		10	11	9	11	6	19	10	7	5	2
Sub-Saharan Africa		SSA	22	22	22	27	10	40	28	19	12	6
Eastern & Southern Africa		ESA	16	15	17	19	8	29	20	13	10	5
West & Central Africa		WCA	28	29	27	38	12	52	36	24	15	7
Least developed countries		LDC	22	22	21							
World			11	11	10	15	6	23	15	10	7	3

Figura 1.1: Porcentaje de niños fuera del sistema escolar primario, según región y grupo socio-económico.

1.2. Estructura de la memoria

A continuación se presenta la estructura de la memoria, organizada en los siguientes capítulos:

- En el capítulo 1 se realiza una introducción al proyecto.
- En el capítulo 2 se muestran los objetivos generales y específicos.
- En el capítulo 3 se muestran todas las tecnologías utilizadas durante el desarrollo del proyecto, desde la más importantes y con mas peso, hasta las auxiliares. Se da una descripción de la tecnología, junto con algunos ejemplos y el porque de su uso en el proyecto.
- En el capítulo 4 se muestra la planificación temporal junto con todos los sprints, donde se explica los objetivos de cada uno y el como fue implementando.
- En el capítulo 5 se describen las distintas pruebas realizadas para comprobar el correcto funcionamiento de la aplicación, así como otros experimentos complementarios.
- En el capítulo 6 se destacan los logros alcanzados a lo largo del proyecto y se plantean soluciones o explicaciones respecto a los objetivos que no pudieron cumplirse.

Capítulo 2

Objetivos

2.1. Objetivo general

Mi trabajo fin de grado consiste en desarrollar una página web educativa interactiva y gratuita, orientada a la enseñanza de las divisiones a niños, que proporcione un entorno seguro, accesible y adaptado al ritmo individual de aprendizaje, mediante el uso de un chatbot conversacional.

2.2. Objetivos específicos

A continuación, se detallan los objetivos específicos planteados para el desarrollo de la página web educativa:

- Desarrollar e integrar un chatbot educativo, entrenado con tecnologías de procesamiento del lenguaje natural, capaz de enseñar y guiar al usuario en el aprendizaje de divisiones, tanto en la practica como en la teoría.
- Diseñar e implementar la interfaz de usuario, con la utilización de herramientas de desarrollo front-end modernas, garantizando así una experiencia visual atractiva y adaptada al público infantil, usando colores vivos, iconos intuitivos, tipografía legible.
- Adaptar los niveles de dificultad del contenido, para poder ajustarse al ritmo de aprendizaje de cada usuario, incluyendo ejercicios de divisiones exactas y no exactas.

- Asegurar que la plataforma funcione en un entorno digital seguro, accesible y gratuito, sin necesidad de registro ni descarga, protegiendo los datos del usuario infantil.
- Evaluar el funcionamiento del sistema mediante pruebas funcionales, verificando la eficacia del chatbot y la usabilidad de la plataforma.

Capítulo 3

Tecnologías utilizadas

3.1. Tecnologías principales

3.1.1. Rasa

RASA[4, 5] es una empresa de software, con central en Berlín que ofrece soluciones para crear chatbots de forma Open-Source y en Python. Al ser Open-Source, proporciona muchas ventajas respecto a otras alternativas, ya que los datos del usuario que usa el asistente no pasan por terceros, y al disponer del framework localmente instalado, aunque el software deje de mantenerse, los chatbots pueden funcionar sin problemas. Además, ofrece una gran flexibilidad para integrarse con otras herramientas y sistemas. Está basado en técnicas de inteligencia artificial y procesamiento del lenguaje natural (PLN). Esta plataforma permite diseñar, entrenar y desplegar chatbots altamente personalizados, capaces de interpretar y responder a las consultas de los usuarios en lenguaje natural. En este trabajo fin de grado, Rasa ha sido la herramienta seleccionada para desarrollar el chatbot educativo. Gracias a sus capacidades de procesamiento del lenguaje natural y gestión del diálogo, permite generar ejercicios personalizados, adaptar el contenido al nivel de dificultad del usuario y ofrecer retroalimentación inmediata. Esto lo convierte en una solución ideal para crear experiencias de aprendizaje interactivas y dinámicas, especialmente adaptadas al público infantil. El sistema se estructura en dos componentes principales:

- Rasa NLU (Natural Language Understanding), encargado de identificar las intenciones del usuario y extraer entidades relevantes de sus mensajes, además dispone de los idiomas

disponibles en la librería spaCy

- Rasa Core, esta parte se encarga de la gestión del diálogo, se conecta con RASA NLU. El componente decide qué acciones tomar en cada momento haciendo uso de un modelo de *machine learning* creado con *tensorflow* y *keras* (herramientas de programación orientadas a la inteligencia artificial). Gestiona la dinámica conversacional determinando las respuestas adecuadas en función del contexto y de la trayectoria previa de la interacción.

En cuanto a la arquitectura de Rasa, este se organiza mediante diversos componentes, agrupados en proyectos, *data/* para los datos de entrenamiento, *models/* para los modelos entrenados, y *actions/* para las acciones personalizadas.

- **nlu.yml**: alojado en la carpeta de *data*, este archivo contiene los datos de entrenamiento para el procesamiento del lenguaje natural (NLU) de Rasa. Aquí es donde definimos las intenciones y entidades que el bot debe reconocer.
- **domain.yml**: en este archivo se especifican los *intents*, *entities*, *slots* y *actions* que definen el comportamiento del chatbot. Los *entities* representan fragmentos específicos de información extraídos de las entradas del usuario, que poseen un valor semántico relevante para la interpretación de la intención del mensaje. Estos elementos permiten al modelo contextualizar y adaptar sus respuestas de forma precisa, facilitando la gestión de diálogos personalizados y dinámicos.

Los *slots* actúan como variables de memoria dentro del sistema conversacional, permitiendo almacenar y conservar información relevante a lo largo del diálogo. Estos elementos son fundamentales para mantener el estado de la conversación, ya que permiten al chatbot recordar datos previamente mencionados por el usuario y utilizarlos en futuras interacciones, optimizando así la coherencia y personalización de las respuestas.

Las *actions* representan las operaciones que el chatbot ejecuta como respuesta a una determinada entrada del usuario y al estado actual de los *slots*. Estas acciones pueden consistir en respuestas simples predefinidas, generación dinámica de texto, consultas a bases de datos externas, o ejecución de lógica personalizada mediante funciones programadas. Las *actions* constituyen, por tanto, el mecanismo a través del cual el modelo traduce la interpretación del lenguaje natural en comportamientos concretos.

- **rules.yml**: este archivo contiene reglas que establecen comportamientos deterministas o respuestas específicas que deben activarse ante determinadas condiciones o intenciones, independientemente del historial de conversación.
- **stories.yml**: en este archivo, definimos los caminos de conversación y las respuestas del bot en función de las acciones que hemos creado, generando así flujos conversacionales, permitiendo al modelo aprender patrones de interacción basados en la experiencia previa de diálogo, facilitando respuestas contextuales más naturales.
- **actions.py**: archivo donde se escribirán las acciones específicas que realizará el chatbot.
- **config.yml**: especifica la configuración del modelo de Rasa, incluyendo los componentes de procesamiento de lenguaje natural y los algoritmos de gestión del diálogo a utilizar.
- **Modelos entrenables**: donde se irán guardando los modelos entrenados.
- **config.yml**: este archivo define la configuración del modelo de Rasa, incluyendo el lenguaje, los componentes del pipeline de procesamiento del lenguaje natural (NLU), y las políticas de gestión del diálogo (Core). El archivo está configurado para trabajar en español, y emplea un conjunto de componentes como *EntitySynonymMapper*, *WhitespaceTokenizer*, y *DIETClassifier*, entre otros. Estos módulos permiten al chatbot comprender mejor la entrada del usuario, detectar entidades, reconocer sinónimos y seleccionar la respuesta adecuada. También incluye el uso de clasificadores como *ResponseSelector* y *FallbackClassifier*, que ayudan a determinar respuestas apropiadas o manejar entradas ambiguas o no reconocidas. Además, se definen las políticas de diálogo que orientan cómo debe comportarse el asistente frente a diferentes situaciones, como *MemorizationPolicy*, que permite recordar conversaciones anteriores, *RulePolicy* que sigue reglas explícitas definidas por el usuario y *TEDPolicy* que aprende patrones de interacción más complejos con base en el contexto y la historia del diálogo.

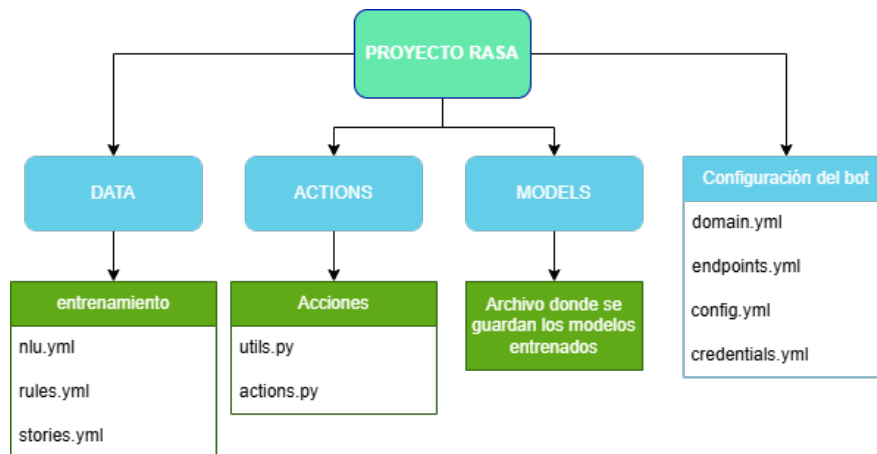


Figura 3.1: Arquitectura de RASA.

3.1.2. React

React¹ es una biblioteca de JavaScript orientada al desarrollo de aplicaciones web y móviles. A diferencia de un framework completo, React se centra exclusivamente en la capa de vista de una aplicación, encargándose del renderizado de los componentes de la interfaz de usuario.

Esta biblioteca permite construir interfaces mediante piezas individuales denominadas componentes, los cuales son funciones de JavaScript que reciben entradas (props) y devuelven elementos que describen qué debe aparecer en pantalla. La organización basada en componentes favorece la reutilización de código, facilita la gestión del estado de la aplicación y mejora la escalabilidad del desarrollo.

Al estar basado en un lenguaje de programación interpretado como JavaScript, no es necesario compilar los programas para ejecutarlos, ya que estos se pueden probar directamente en cualquier navegador sin necesidad de procesos intermedios.

Debido a todo ello, React es el encargado de hacer posible que la interfaz del chatbot sea interactiva, clara y adaptable, permitiendo mostrar los mensajes, opciones y ejercicios de forma atractiva.

¹<https://es.react.dev/>

3.1.3. JavaScript

JavaScript[6] es un lenguaje de programación de alto nivel utilizado principalmente para dotar de dinamismo e interactividad a las páginas web. Las páginas dinámicas permiten incorporar efectos sobre el contenido textual, animaciones, acciones que se desencadenan al interactuar con elementos de la interfaz, como botones, así como la generación de ventanas emergentes de aviso dirigidas al usuario.

Los navegadores web modernos incluyen soporte para JavaScript, que regula las especificaciones del lenguaje. Sin embargo, existen diferencias de implementación entre los navegadores: mientras que Internet Explorer utiliza una variante denominada JScript, otros navegadores como Firefox, Opera, Safari y Konqueror implementan JavaScript conforme a las directrices del estándar.

Por este motivo, JavaScript ha sido fundamental en el desarrollo del proyecto, ya que permite construir interfaces web dinámicas e interactivas que mejoran la experiencia del usuario y facilitan la comunicación fluida con el asistente conversacional.

3.1.4. HTML5

HTML5[7, 8] (HyperText Markup Language, versión 5) es el lenguaje de marcado de hipertexto considerado como el estándar actual para la creación de sitios web. Utiliza un sistema de etiquetas de apertura y cierre que permite estructurar y mostrar información en la web de manera eficiente.

Con la aparición de HTML5, junto a tecnologías asociadas como CSS3 y JavaScript, el desarrollo web ha experimentado una profunda transformación. Ya no se limita únicamente a interconectar documentos, sino que permite la creación de aplicaciones interactivas, dinámicas y adaptadas a la economía digital actual. Además, los nuevos estándares han introducido métodos de desarrollo más ágiles y participativos, incorporando el feedback de usuarios y comunidades en el proceso de estandarización. Por ello, HTML5 ha sido utilizado en este proyecto como base para estructurar la interfaz del chatbot, permitiendo una presentación clara, intuitiva y semánticamente organizada del contenido educativo.

3.1.5. CSS

CSS[9] (Cascading Style Sheets) es un lenguaje de programación de código abierto utilizado en la construcción de sitios web y en plantillas HTML, que integra toda la información relevante relacionada con la visualización de páginas web.

CSS se utiliza para dar formato a la apariencia y estructura de una página web, así como para establecer características de diseño como la distribución básica, los colores y las fuentes.

CSS permite mantener la continuidad entre diferentes páginas de un mismo sitio web y facilita y acelera el desarrollo de páginas web. Además, CSS es uno de los lenguajes base de la *Open Web* y posee una especificación estandarizada por parte del W3C. Por ello, CSS ha sido utilizado en este proyecto para diseñar una interfaz visualmente atractiva, coherente y adaptada al público infantil, mejorando la usabilidad del chatbot y reforzando el enfoque educativo.

3.1.6. Intro.js

Intro.js[10] es una librería de código abierto escrita en JavaScript puro y CSS, utilizada para implementar un recorrido guiado por la interfaz, paso a paso.

La simplicidad de la API de Intro.js facilita el desarrollo de un recorrido de incorporación (onboarding) avanzado. Intro.js no requiere ninguna dependencia. Todo lo que necesitas hacer es añadir los archivos JS y CSS. Por ello, ha sido empleada en este trabajo para guiar al usuario en su primera interacción con la interfaz, facilitando la comprensión del funcionamiento del chatbot de forma visual e intuitiva.

3.1.7. Python

Python[11] fue creado a principios de los años 90 por Guido van Rossum, y su nombre proviene de la serie de humor “Monty Python’s Flying Circus”. Es un lenguaje de programación de alto nivel, gratuito, de código abierto y que cuenta con un extenso paquete de bibliotecas estándar, lo que facilita el desarrollo rápido de aplicaciones en diversos ámbitos.

El principal objetivo de Python es promover la claridad y la legibilidad del código, favoreciendo una sintaxis sencilla y similar al lenguaje natural. Python soporta múltiples paradigmas de programación, entre ellos el orientado a objetos, el imperativo y el funcional.

Además, Python es un lenguaje con tipado dinámico, lo que significa que no es necesario

declarar de forma explícita el tipo de los datos. El propio intérprete deduce el tipo en función del valor introducido por el usuario o el programador, permitiendo así un desarrollo más ágil y flexible.

Gracias a estas características, junto con su independencia de plataforma y su facilidad de aprendizaje, Python se ha convertido en uno de los lenguajes más populares y utilizados en ámbitos tan variados como el desarrollo web, la inteligencia artificial, la ciencia de datos o la automatización de procesos.

Por estas razones, Python ha sido el lenguaje utilizado en este proyecto, dado que Rasa requiere su uso para la implementación de acciones personalizadas. Su sintaxis clara y su amplia comunidad facilitan el desarrollo, la depuración y la ampliación de funcionalidades, aspectos clave para garantizar la calidad y escalabilidad del chatbot educativo.

3.1.8. Json

JSON[12] (JavaScript Object Notation) es un formato estándar utilizado para el intercambio de datos entre un servidor y un navegador, o entre diferentes aplicaciones. Fue desarrollado por Douglas Crockford.

JSON permite enviar y recibir datos en texto plano de manera sencilla y eficiente, siendo fácil de leer tanto para personas como para sistemas automatizados. Cuando se transmiten datos utilizando este formato, se convierten a texto plano, y posteriormente se reconvierten al formato adecuado, como JavaScript, si se trabaja desde este lenguaje.

Una de las principales ventajas de JSON es su enfoque en el envío y la recepción de datos de forma ligera y estructurada. Además, JSON es independiente del lenguaje de programación, lo que permite su utilización tanto en entornos de programación orientados a objetos, como en tecnologías del lado del servidor o en lenguajes de manipulación de bases de datos.

Esta versatilidad y facilidad de integración han convertido a JSON en uno de los formatos de intercambio de datos más utilizados en el desarrollo de aplicaciones web modernas.

Por ello, JSON ha sido empleado en este proyecto para estructurar y transmitir los datos entre la interfaz desarrollada en React y el backend gestionado por Rasa, facilitando una comunicación clara, ligera y estandarizada.

3.1.9. Spacy

SpaCy[13] es una biblioteca avanzada de Procesamiento del Lenguaje Natural (PLN) escrita en Python, que destaca por su eficiencia y velocidad. En este proyecto, SpaCy se integra dentro del pipeline de NLU de RASA para realizar tareas cruciales en la comprensión del lenguaje del usuario. Específicamente, SpaCy se utiliza para:

- Tokenización: Dividir las preguntas y respuestas de los niños en unidades individuales (tokens) para su posterior análisis.
- Lematización: Reducir las palabras a su forma base, lo que ayuda a RASA a identificar la intención del usuario independientemente de las variaciones verbales o nominales.
- Etiquetado Morfosintáctico: Asignar categorías gramaticales a cada palabra, proporcionando información contextual adicional a RASA.

La utilización de SpaCy permite que el chatbot comprenda de manera más efectiva las preguntas relacionadas con las divisiones, identificando los números, las operaciones y la intención del usuario para poder ofrecer una respuesta o guía adecuada. Por estas razones, SpaCy ha sido la herramienta elegida para optimizar el análisis lingüístico del lenguaje en este proyecto, mejorando la precisión del reconocimiento de intenciones y la adaptación del diálogo.

3.2. Tecnologías auxiliares

3.2.1. GitHub

GitHub[14] es una plataforma en línea que funciona como servidor de alojamiento para proyectos gestionados mediante Git, permitiendo tanto la colaboración entre múltiples usuarios como el trabajo individual. Un repositorio es una carpeta donde se desarrolla un proyecto y que almacena todos los archivos necesarios para su ejecución. Aunque existen otras plataformas con funciones similares, GitHub se ha consolidado como la más popular en la actualidad.

GitHub facilita el registro del progreso de los proyectos de manera remota, el intercambio de proyectos entre usuarios, y ofrece la seguridad y accesibilidad propia de la nube. En proyectos colaborativos, la premisa básica es que todos los participantes reconocen el repositorio alojado

en GitHub como la copia principal del proyecto, centralizando el control de versiones, aunque Git esté diseñado como un sistema distribuido.

Para comprender cómo Git administra los cambios y cómo se comparten entre colaboradores, es fundamental conocer su estructura y su sincronización con GitHub. El flujo de trabajo se organiza en cuatro zonas principales:

- Directorio de trabajo (working directory): es la carpeta local donde se realizan las modificaciones activas.
- Área de preparación (staging area o index): es el espacio intermedio donde se seleccionan los archivos que se incluirán en el próximo registro o “commit”.
- Repositorio local (local repository o HEAD): almacena en el equipo todos los cambios confirmados.
- Repositorio remoto (remote repository): almacena en la nube (por ejemplo, en GitHub) los cambios sincronizados desde el repositorio local.

Se ha elegido esta plataforma debido a su amplia adopción dentro de la comunidad de desarrolladores y a la familiaridad adquirida con su uso a lo largo de la carrera. Además, ha permitido gestionar el código del proyecto de forma estructurada, accesible, eficiente y segura, aspectos fundamentales para el correcto desarrollo del trabajo.

3.2.2. Visual Studio Code

Visual Studio Code[15] es un editor de código fuente desarrollado por Microsoft, gratuito, de código abierto y multiplataforma. Está disponible para Windows, macOS, Linux, Raspberry Pi OS y versión web. Visual Studio Code es conocido por su flexibilidad y capacidad de ampliación, lo que lo convierte en una herramienta ideal para desarrolladores de todos los niveles. Es compatible con una amplia variedad de lenguajes de programación y plataformas, y ofrece una experiencia de desarrollo altamente personalizable gracias a su amplio ecosistema de extensiones.

En este proyecto, Visual Studio Code ha sido utilizado como entorno de desarrollo principal, ya que su integración con Git, sus extensiones específicas para Python, Rasa y React, y su

interfaz intuitiva han facilitado la edición, organización y prueba del código de manera eficiente y centralizada.

3.2.3. LaTeX

LaTeX[16] es un sistema de software totalmente abierto, disponible de forma gratuita, enfocado en la preparación de documentos, ampliamente utilizado en los campos de las matemáticas y las ciencias naturales, pero que también se está extendiendo a otras disciplinas.

Un documento en LaTeX consiste en uno o más archivos fuente que contienen texto en formato plano, el contenido textual real más comandos de marcado. Estos incluyen instrucciones que permiten insertar material gráfico producido por otros programas.

Para la elaboración de esta memoria se ha utilizado la tecnología LaTeX, por su capacidad para generar documentos técnicos de alta calidad, su precisión tipográfica y su eficacia para estructurar textos académicos extensos de manera clara y profesional. Esta se ha combinado con Overleaf, una plataforma de edición en línea que simplifica la escritura y compilación de documentos, permitiendo al usuario centrarse en el contenido sin preocuparse por el formato.

Capítulo 4

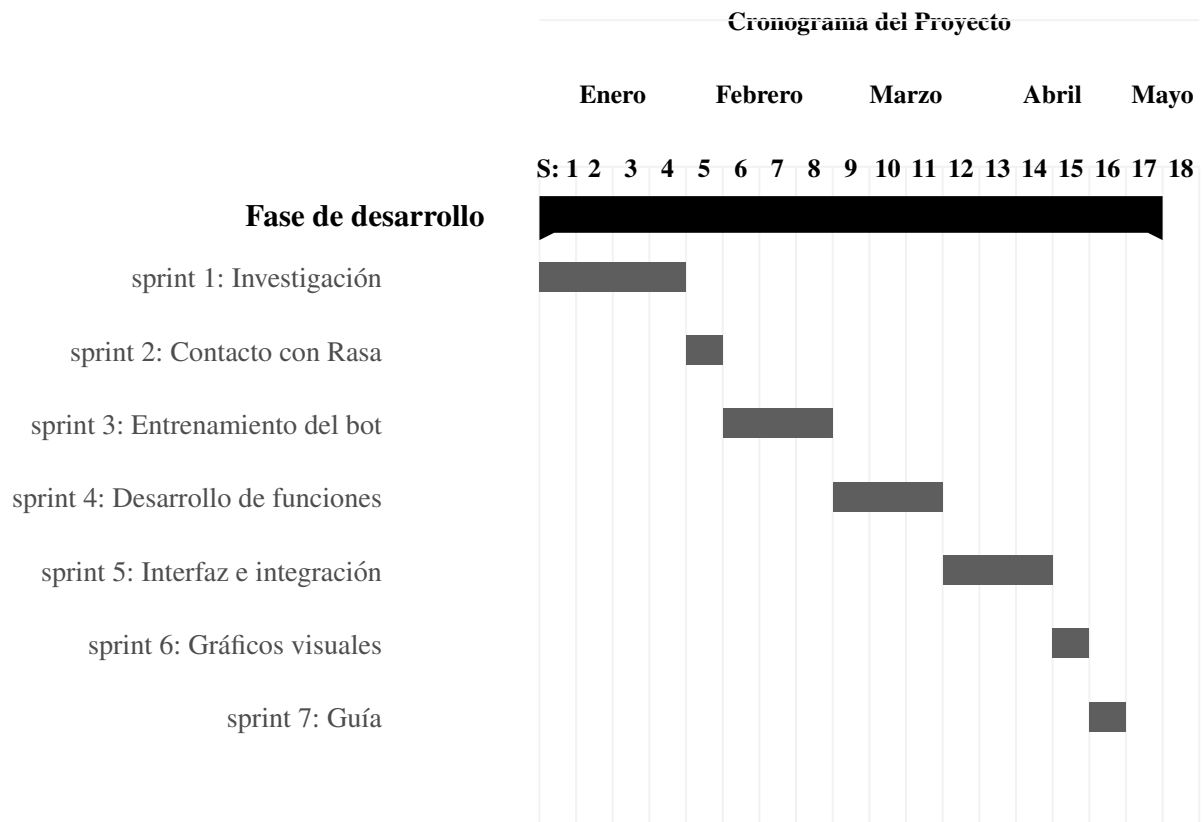
Diseño e implementación del proyecto

4.1. Planificación temporal

El proyecto se inició como una propuesta en septiembre de 2024, pero no fue hasta diciembre cuando Jesús García Parrado aprobó el tema.

Se ha trabajado en sprints de aproximadamente dos semanas; sin embargo, durante ese periodo fue necesario realizar pausas debido a compromisos externos al entorno educativo. En los primeros meses, especialmente hasta marzo, el desarrollo se centró principalmente en fines de semana, ya que no se disponía de tiempo suficiente entre semana. No fue hasta marzo cuando se logró dedicar también algunos días laborales, lo que permitió avanzar de forma más constante. En particular, durante todo el mes de enero, el sprint se centró en la investigación y pruebas para identificar las tecnologías más adecuadas para el desarrollo del proyecto.

En el siguiente diagrama de Gantt se puede observar el tiempo exacto dedicado a cada sprint. Los sprints tercero, cuarto y quinto fueron los más extensos, debido a la complejidad técnica que implicaba cada una de sus fases. En cambio, los dos últimos sprints fueron más breves, ya que se enfocaron únicamente en mejoras orientadas a la experiencia del usuario.



Nota: Diagrama de Gantt.

4.2. Sprint 1: Investigación sobre las tecnologías a utilizar

El primer Sprint fue la búsqueda y selección de tecnologías necesarias para el desarrollo del proyecto, buscando las que mejor se adaptaran a los requisitos específicos.

4.2.1. Objetivos

Durante la realización del sprint se fijaron los siguientes objetivos:

- Investigación de frameworks adecuados para el entrenamiento del chatbot.
- Búsqueda alternativas para el desarrollo del front-end.

4.2.2. Implementación del desarrollo

Con los objetivos del sprint definidos, se inició un análisis comparativo de distintas tecnologías, considerando criterios como el control sobre el entrenamiento del chatbot, la facilidad de aprendizaje, la disponibilidad de documentación, la frecuencia de actualización, el soporte comunitario y la capacidad de integración.

Como resultado de este proceso, se seleccionaron las tecnologías Rasa y React para el desarrollo del asistente conversacional y de la interfaz web, respectivamente. Los motivos detallados de esta elección se recogen en el apartado 3.1 de esta memoria.

4.3. Sprint 2: Primeros pasos en RASA

Este Sprint es la primera toma de contacto con RASA, explorando sus características fundamentales y comprendiendo su estructura básica.

4.3.1. Objetivos

Los objetivos de este sprint serán los siguientes:

- Asegurar su correcto despliegue y funcionamiento en el entorno de desarrollo.
- Comprender su funcionamiento.
- Que tenga la capacidad de responder a preguntas comunes como saludos y despedidas.

4.3.2. Implementación del desarrollo

Una vez seleccionado el framework para el desarrollo del chatbot, se instaló en el entorno local usando el siguiente comando en la terminal:

```
pip install rasa
```

Al iniciar RASA por primera vez, el entorno genera automáticamente un proyecto básico pre-configurado que incluye los componentes esenciales mínimos, como archivos de entrenamiento, configuración y flujo de diálogo, permitiendo así el despliegue inmediato de un chatbot funcional sin necesidad de configuraciones adicionales iniciales. Para poder comprobar si se realiza con éxito el despliegue de RASA se usaron los siguientes comandos en la terminal de VSC.

```
rasa run
```

Abriendo así el chatbot en la misma terminal. Además del despliegue inicial, se exploraron los archivos generados por defecto (como `nlu.yml`, `domain.yml` y `stories.yml`) para comprender cómo se estructura internamente el chatbot y cómo se configuran las intenciones básicas.

Como parte de las pruebas, se definieron intenciones simples como saludos y despedidas, verificando que el bot pudiera responder correctamente a expresiones como “hola” o “adiós”.

4.4. Sprint 3: Entrenamiento del bot

En este Sprint se trabajó en el entrenamiento del chatbot. Para ello, se amplió el archivo `nlu.yml` incorporando nuevas intenciones y múltiples ejemplos de expresiones que los usuarios podrían utilizar. Además, se definieron entidades específicas para reconocer valores numéricos y operaciones básicas.

4.4.1. Objetivos

- Incorporación de nuevas intenciones
- Creación de entidades.
- Definir reglas y diseñar historias adicionales

4.4.2. Implementación del desarrollo

NLU.YML

Se diseñaron e incorporaron nuevos *intents*, específicos para las preguntas relacionadas con las divisiones. Además de añadir más ejemplos para cada intención preestablecida.

En cuanto a preguntas relacionadas por conocer la teoría de las divisiones se crearon los siguientes *intents*:

```
division_type  
choose_one_digit_division  
choose_two_digit_division
```

```
choose_decimal_division  
choose_division_with_remainder
```

Para aquellas preguntas que requieran generación de ejercicios se crearon los siguientes:

```
request_practice_div  
choose_number_of_exercises  
choose_individual_divisions  
choose_number_of_exercises  
answer_division  
choose_difficulty  
choose_statement_difficulty
```

Y por último, aquellas preguntas que requerían de una solución matemática se crearon:

```
ask_sum  
ask_div_int
```

DOMAIN.YML

Al tener bien definidos los intents en nlu.yml, el siguiente paso es establecer las respuestas del bot en domain.yml.

```
utter_ask_sum  
utter_ask_div  
utter_greet  
utter_ask_wellbeing  
utter_mood_unhappy  
utter_happy  
utter_goodbye  
utter_iamabot  
utter_result_sum  
utter_result_difference  
utter_out_of_scope  
utter_thank_you  
utter_fallback
```

Asimismo, se implementaron respuestas con opciones predefinidas que permiten al usuario seleccionar entre distintas alternativas, facilitando así una interacción más guiada y estructurada dentro del flujo conversacional.

```
utter_division_type
utter_choose_one_digit_division
utter_choose_two_digit_division
utter_choose_decimal_division
utter_choose_division_with_remainder
utter_request_practice_div
utter_choose_number_of_statement_exercises
utter_choose_number_of_exercises
utter_elegir_nivel_dificultad
utter_elegir_nivel_dificultad_enunciados
```

ENTIDADES

Las entidades creadas fueron:

- `number1` y `number2`: utilizadas para identificar los operandos en las divisiones propuestas por el usuario.
- `num_ejercicios`: entidad que captura la cantidad de ejercicios que el usuario desea practicar.
- `numero`: utilizada para capturar la cantidad de ejercicios que el usuario desea realizar, tanto en divisiones con enunciado como sin enunciado.
- `dificultad`: permite identificar el nivel de dificultad solicitado por el usuario (fácil, medio, difícil).

Estas entidades fueron integradas en las distintas intenciones del bot, facilitando una interacción más personalizada y precisa.

STORIES.YML

En este archivo se definieron los flujos conversacionales en los que el usuario solicita aprender la teoría, y el chatbot responde mediante opciones interactivas presentadas en forma de botones. Cada botón permite seleccionar uno de los cuatro tipos de divisiones disponibles, y al ser

pulsado, se activa una acción de tipo utter, encargada de proporcionar una respuesta predefinida que introduce el tipo de división elegido y guía al usuario en el proceso de aprendizaje. Como se puede observar en el siguiente fragmento de código, cada intent tiene una acción definida.

```
- story: usuario quiere aprender a dividir
  steps:
    - intent: division_type
    - action: utter_division_type

- story: usuario elige division de una cifra
  steps:
    - intent: choose_one_digit_division
    - action: utter_choose_one_digit_division

- story: usuario elige division de dos cifras
  steps:
    - intent: choose_two_digit_division
    - action: utter_choose_two_digit_division

- story: usuario elige division con decimales
  steps:
    - intent: choose_decimal_division
    - action: utter_choose_decimal_division

- story: usuario elige division con residuo
  steps:
    - intent: choose_division_with_remainder
    - action: utter_choose_division_with_remainder
```

Este fragmento representa un ejemplo ilustrativo de los distintos flujos conversacionales desarrollados para las actividades de práctica de divisiones sueltas. El proceso se inicia cuando el usuario expresa la intención de practicar, a lo que el chatbot responde con una acción que introduce la actividad. A continuación, el usuario elige practicar divisiones individuales, y el sistema solicita que seleccione un nivel de dificultad, utilizando la entidad dificultad para registrar esta

elección. Tras ello, se solicita la cantidad de ejercicios a realizar, que se captura mediante la entidad número.

Una vez definidos estos parámetros, se ejecuta la acción personalizada `action_generate_exercises` que genera dinámicamente los ejercicios de división. Finalmente, el usuario responde a cada división mediante la intención `answer_division`, y el chatbot evalúa su respuesta a través de la acción personalizada `action_check_division_answer`, proporcionando retroalimentación inmediata.

```
- story: usuario quiere practicar a divisiones sueltas
steps:
  - intent: request_practice_div
  - action: utter_request_practice_div
  - intent: choose_individual_divisions
  - action: utter_elegir_nivel_dificultad
  - intent: choose_difficulty
    entities:
      - dificultad: dificultad
  - action: utter_choose_number_of_exercises
  - intent: choose_number_of_exercises
    entities:
      - numero
  - action: action_generate_exercises
  - intent: answer_division
  - action: action_check_division_answer
```

Una vez definidos todos los flujos conversacionales y desarrolladas las acciones personalizadas, las cuales al estar en la etapa inicial eran muy simples y tenían como objetivo únicamente verificar el funcionamiento básico del flujo, se procedió al entrenamiento del chatbot. Tras completar el entrenamiento, es necesario desplegar nuevamente el sistema.

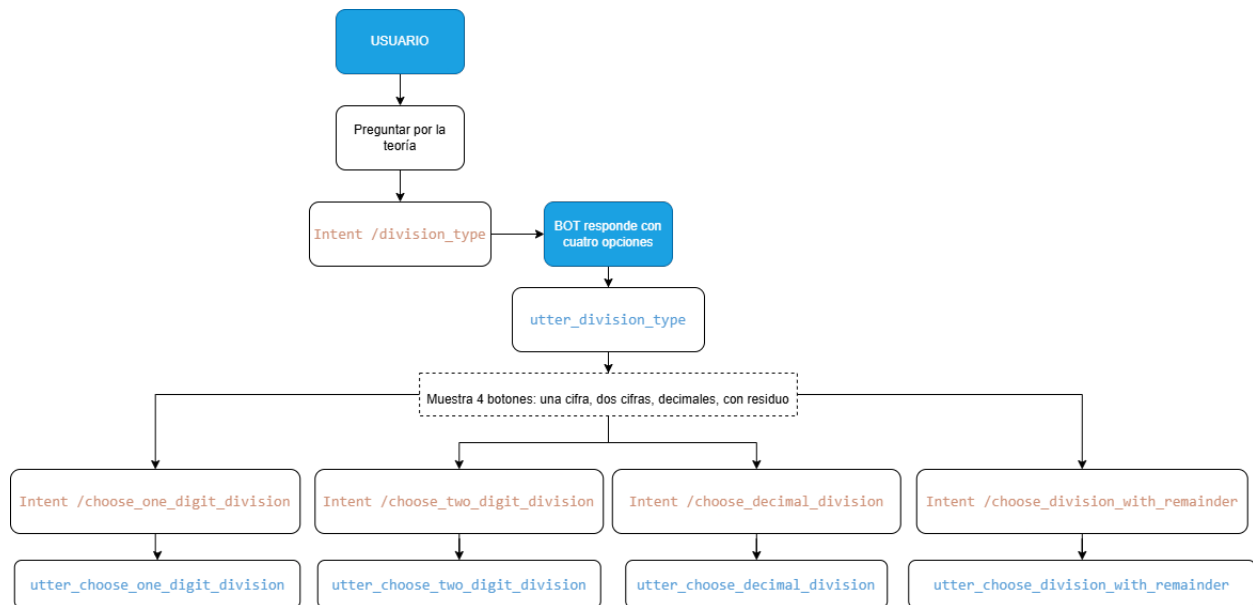


Figura 4.1: Flujo conversacional

4.5. Sprint 4: Desarrollo de funcionalidades del bot

Durante este Sprint se desarrollaron las funciones que permitirán al chatbot interactuar de forma dinámica con el usuario. Para ello, se implementaron acciones personalizadas en el archivo `actions.py`, tales como la generación de divisiones aleatorias, la validación de respuestas y la entrega de mensajes de retroalimentación.

4.5.1. Objetivos

Los objetivos que se plantearon para este sprint fueron los siguientes:

- Diseñar, implementar y probar las funciones personalizadas que dotan al chatbot de la capacidad de generar ejercicios de división de forma dinámica.
- Adaptar los ejercicios según distintos niveles de dificultad, incluyendo variantes con enunciados y sin enunciados, para atender distintos estilos de aprendizaje.
- Asegurar que la lógica de validación de respuestas sea precisa y robusta.
- Integrar mensajes de retroalimentación específicos para mejorar la experiencia de aprendizaje, reforzar los aciertos y corregir los errores cometidos por el usuario.

4.5.2. Implementación del desarrollo

Durante el desarrollo de este sprint se decidió separar las acciones personalizadas en dos archivos: `utils.py` y `actions.py`, con el objetivo de organizar mejor el código y facilitar su mantenimiento.

UTILS.PY

- `generate_division`: genera divisiones de acuerdo con el nivel de dificultad seleccionado por el usuario.
- `generate_enunciado`: selecciona un enunciado del archivo `enunciados.json` en función de la dificultad elegida, complementando el ejercicio generado.
- `load_enunciados`: carga y devuelve la lista de enunciados almacenados en el archivo `enunciados.json`.
- `explicar_divisiones_incorrectas`: crea un mensaje explicativo para el usuario cuando comete errores en los ejercicios de división. Además, genera una lista de divisiones simples que pueden reutilizarse desde la interfaz desarrollada en React.

ACTIONS.PY

En el archivo `actions.py` se importan varios módulos, donde `rasa_sdk` permite definir acciones personalizadas en el asistente conversacional, como `Action` y `Tracker`. También se importan funciones definidas en el módulo `utils.py`.

```
from typing import Any, Text, Dict, List

import random
import os
import json
from rasa_sdk import Action, Tracker
from rasa_sdk.executor import CollectingDispatcher
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher
from rasa_sdk.events import FollowupAction
```

```
from actions.utils import generate_division,  
    explicar_divisiones_incorrectas, generate_enunciado
```

En actions.py se implementaron cuatro funciones:

- **ActionSum**: Esta función se activa cuando el usuario pregunta por el resultado de una suma. Para ello, extrae los valores numéricos del mensaje mediante los *slots* `number1` y `number2`, los cuales han sido definidos previamente en el archivo `domain.yml`.

```
first_number =  
    next(tracker.get_latest_entity_values("number1"), None)  
second_number =  
    next(tracker.get_latest_entity_values("number2"), None)
```

Una vez recuperados, ambos valores se convierten a tipo `float` para permitir operaciones con números decimales.

- **ActionDiv**: Función que se activa cuando el usuario pide el resultado de una división. Al igual que en la acción **ActionSum** también se extraen los valores numéricos, guardándolos en los *slots* `number1` y `number2`. Dentro de esta función se verifica que el divisor no sea 0, en cuyo caso se enviará un mensaje explicando que no se puede dividir entre 0. Una vez comprobado que la división sí existe se enviará un mensaje predefinido, donde se le da la solución con un ejemplo.
- **ActionGenerarEjercicios**: Esta función genera una serie de ejercicios de división personalizados según los *slots* proporcionados por el usuario: dificultad, cantidad y tipo de ejercicio. Si falta alguno de ellos, se aplican valores por defecto o se solicita al usuario que lo corrija.

```
dificultad_ejercicio = tracker.get_slot("dificultad")  
num_ejercicios = tracker.get_slot("numero")  
tipo_ejercicio = tracker.get_slot("tipo_ejercicio")
```

En función del tipo de ejercicio, sueltas o con enunciado, se llaman funciones específicas para generar los ejercicios. Luego, se envía el primer ejercicio al usuario y se almace-

nan en *slots* la lista completa, el índice actual y la respuesta correcta para continuar la interacción.

```
if tipo_ejercicio == "sueeltas":
    ejercicios = [generate_division(dificultad_ejercicio) for _
                  in range(num_ejercicios)]
else:
    ejercicios = [generate_enunciado(dificultad_ejercicio) for
                  _ in range(num_ejercicios)]
```

- **ActionCheckDivisionAnswer:** Esta función valida la respuesta del usuario frente al ejercicio de división actual. Primero recupera la respuesta ingresada, la respuesta correcta y el estado del ejercicio desde los *slots*. Aplica una tolerancia para aceptar pequeños errores de redondeo y, en función del resultado, devuelve un mensaje de acierto o error. Si la respuesta es incorrecta, almacena los datos del ejercicio fallado en una lista. Luego, avanza al siguiente ejercicio o, si se han completado todos, llama a la función `explicar_divisiones_incorrectas` para generar un resumen final con explicaciones y refuerzos visuales.

```
user_answer = float(user_answer_raw.replace(",", ".").strip())
if abs(user_answer - correct_answer) < tolerancia:
    dispatcher.utter_message(text=" Correcto !",
                             custom={"correcto": True})
else:
    incorrect_divisions.append({...})
    dispatcher.utter_message(text="Incorrecto...",
                             custom={"correcto": False})
```

4.6. Sprint 5: Creación de la interfaz web e Integración con Rasa

Este Sprint se centró en el desarrollo de una interfaz web que permite visualizar de forma completa todos los mensajes generados por el bot durante la conversación con el usuario.

4.6.1. Objetivos

Los objetivos de este sprint fueron:

- Desarrollar una interfaz intuitiva y fácil de utilizar, priorizando un diseño visualmente atractivo y adecuado para el público infantil.
- Utilizar tipografía de gran tamaño, diseñada para facilitar la comprensión lectora en personas con dislexia.
- Implementar botones animados que favorezcan la interacción y el aprendizaje de forma lúdica.
- Integrar el front con Rasa

4.6.2. Implementación del desarrollo

Para la creación de la interfaz se optó por un diseño sencillo y funcional, con los logros ubicados a la izquierda y el chat en el centro, donde el usuario puede interactuar con el bot. Los colores elegidos para los mensajes fueron azul para los enviados por el usuario y gris para las respuestas generadas por el bot, favoreciendo así una distinción visual clara entre ambos.

Para la elección de la tipografía se consideraron varias opciones diseñadas para mejorar la lectura en personas con dislexia, siendo la elegida Open Dyslexic [17], diseñada por Abelardo González; esta fuente presenta características morfológicas similares a Dyslexie, aunque fue desarrollada a partir de la fuente Bitstream Vera. A diferencia de otras tipografías, Open Dyslexic es completamente libre para cualquier tipo de uso.

En la figura 4.2 se puede ver la página web.

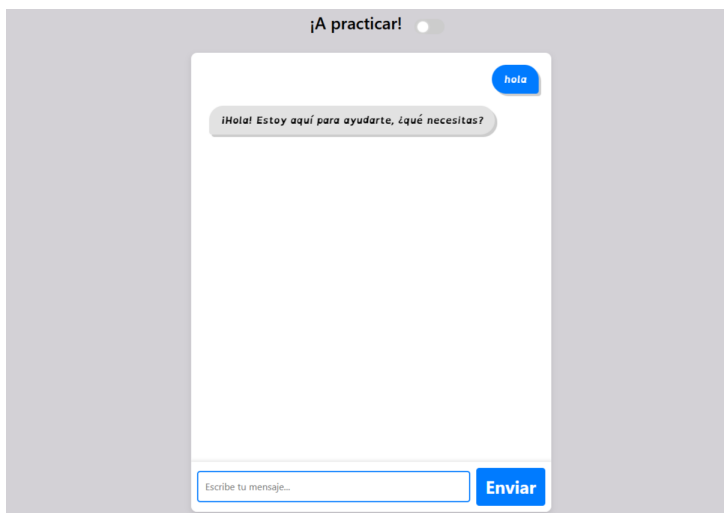


Figura 4.2: Página Web con el chat integrado

Para mejorar la interacción de los usuarios con el chatbot, se modificó el diseño de los botones generados por este, otorgándoles un estilo más llamativo. Además, se incorporaron animaciones visuales: al pasar el cursor por encima, el color del botón se oscurece ligeramente y se produce un efecto que simula su hundimiento, lo que aporta una experiencia más dinámica y atractiva. Ver figura 4.3

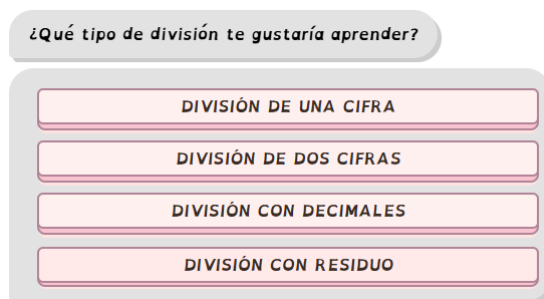


Figura 4.3: Elección de divisiones mediante botones

Para establecer la conexión entre el front-end y Rasa mediante una petición HTTP al webhook REST, se incorporó la siguiente línea de código dentro del componente Chatbot.js.

```
const response = await
  fetch("http://localhost:5005/webhooks/rest/webhook", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ sender: "user", message: messageText })
```



```
} ) ;
```

4.7. Sprint 6: Generación de gráficos para explicar las divisiones

Este Sprint se centró en la realización de gráficos que enviará el bot de todos los ejercicios que el usuario haya fallado.

4.7.1. Objetivos

- Crear gráficos para una mejor comprensión de las divisiones.

4.7.2. Implementación del desarrollo

Se diseñó una función capaz de generar gráficos dinámicos en función de los valores de las divisiones falladas. Estos gráficos utilizan recuadros para representar visualmente el dividendo, dentro de los cuales se colocan bolas que simbolizan el divisor, facilitando así la comprensión del proceso de división por parte del usuario.

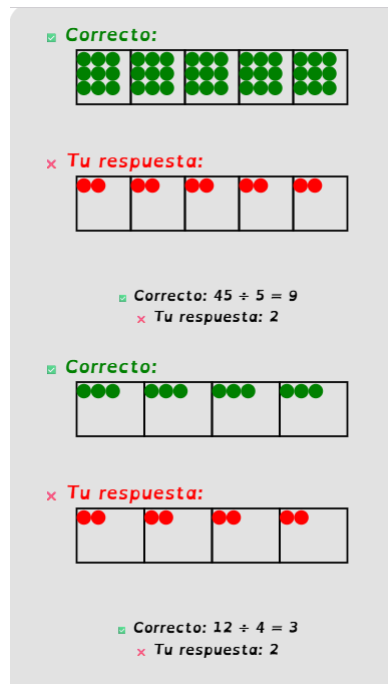


Figura 4.4: Corrección visual.

4.8. Sprint 7: Implementación de una guía de usuario y sistemas de logros

Este Sprint fue el último en realizarse.

4.8.1. Objetivos

- Crear una guía de usuario para la pagina web.
- Añadir un sistema de logros

4.8.2. Implementación del desarrollo

Guía

Para facilitar la comprensión de la interfaz por parte del usuario, se integró una guía interactiva mediante la librería Intro.js. Esta guía no se activa automáticamente, sino que está disponible de forma opcional a través de un botón identificado con el ícono “Ver guía”, ubicado en la

4.8. SPRINT 7: IMPLEMENTACIÓN DE UNA GUÍA DE USUARIO Y SISTEMAS DE LOGROS33

parte derecha de la pantalla. Al hacer clic en dicho botón, se inicia un recorrido que destaca los elementos más relevantes de la interfaz, como el área de logros y los botones de respuesta rápida del bot. Esta decisión de activación manual permite que el usuario acceda a la guía únicamente cuando lo desee, evitando interrupciones innecesarias durante el uso normal de la aplicación.

Logros

En cuanto al sistema de logros, se optó por una representación mediante insignias, con el objetivo de motivar y reforzar positivamente el progreso del usuario. Inicialmente, las insignias aparecen en color negro, indicando que están bloqueadas. A medida que el usuario alcanza ciertos objetivos, como resolver correctamente una cantidad determinada de divisiones, las insignias se desbloquean y se reemplazan por versiones a color, lo que proporciona una retroalimentación visual inmediata y estimulante.

```
<div className="col-md-3 text-center mb-4" data-intro="Aqu  
est n tus logros">  
  <img  
    src={correctAnswers >= 10 ? "logro_10.png" :  
      "sinlogro10.png"}  
    alt="Logro 10"  
    className="img-fluid mb-2"  
  />  
  <img  
    src={correctAnswers >= 50 ? "logro50.png" :  
      "sinlogro10.png"}  
    alt="Logro 50"  
    className="img-fluid mb-2"  
  />  
  <img  
    src={correctAnswers >= 100 ? "logro100.png" :  
      "sinlogro10.png"}  
    alt="Logro 100"  
    className="img-fluid mb-2"  
  />  
</div>
```

Vistazo general a la interfaz

En la Figura 4.5 se muestra la página web. En esta etapa, el usuario aún no ha interactuado con el chatbot ni ha desbloqueado logros.

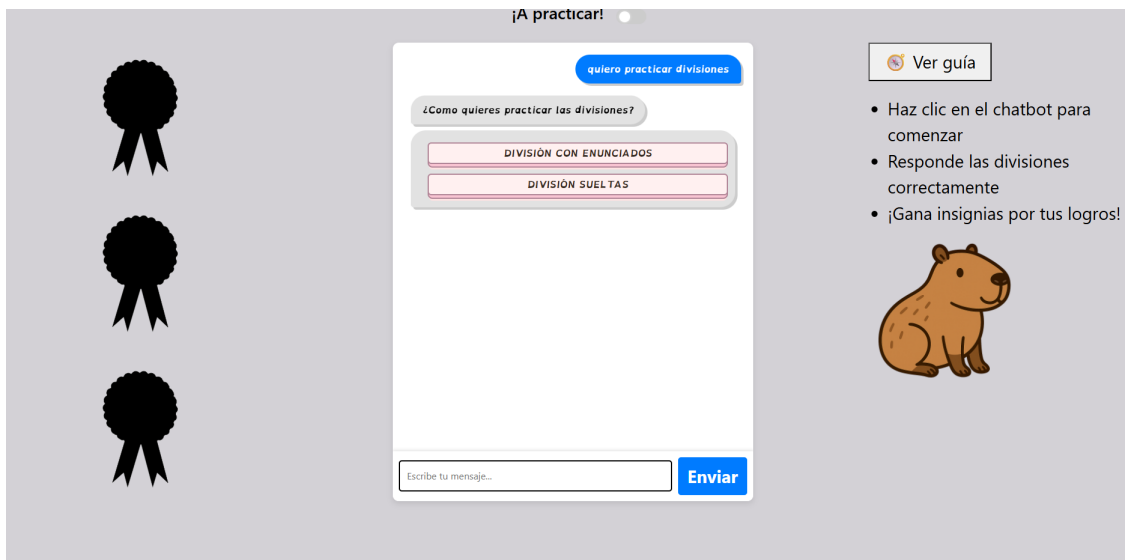


Figura 4.5: Vista inicial de la página web al ingresar

Por otro lado, la Figura 4.6 presenta el aspecto final de la interfaz con el modo oscuro activado. En esta vista se puede observar que el usuario ha alcanzado todos los logros disponibles.

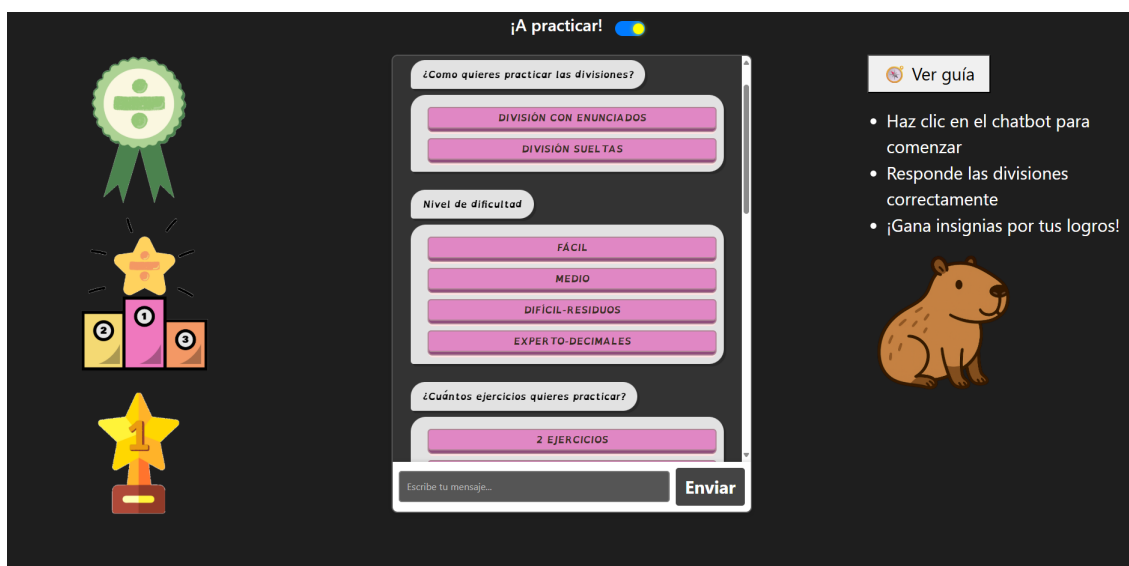


Figura 4.6: Vista final de la página web en modo oscuro activado

4.9. Arquitectura general

El proyecto se basa en una arquitectura cliente-servidor, donde el **front-end** fue desarrollado en *React* y se encarga de gestionar la interfaz de usuario, mientras que el **back-end** está compuesto por un chatbot creado con *Rasa*, el cual se comunica mediante peticiones HTTP al webhook REST que este ofrece.

La comunicación entre ambos componentes se realiza a través de llamadas `fetch()` desde el cliente, enviando los mensajes del usuario al servidor de *Rasa* y recibiendo las respuestas generadas por el bot. El front-end también integra la librería *Intro.js* para ofrecer una guía interactiva, y gestiona un sistema de logros visuales basado en el número de respuestas correctas del usuario.

A nivel visual, el proyecto se organiza en tres columnas principales: logros a la izquierda, chatbot al centro y ayuda/interfaz adicional a la derecha. Además, se implementó un modo oscuro y un control de estado usando *React Hooks* para una experiencia más fluida.

Capítulo 5

Experimentos y validación

En este capítulo se analizarán las pruebas realizadas al chatbot para poder confirmar que funciona según lo esperado. Estas pruebas y validaciones son cruciales en cualquier proyecto, ya que aseguran que nuestra solución propuesta cumple con las expectativas y requerimientos necesarios definidos previamente para validar nuestro bot.

5.1. Reconocimiento del lenguaje natural

Con el objetivo de validar la capacidad del chatbot para interpretar correctamente los mensajes del usuario y generar respuestas coherentes, se llevaron a cabo distintas pruebas centradas en el reconocimiento del lenguaje natural. **Faltas de ortografía**

Estas pruebas consistieron en enviar al bot distintas formulaciones de una misma pregunta o instrucción, con variaciones sintácticas, errores ortográficos intencionales y diferentes niveles de formalidad. El propósito era comprobar si el sistema, mediante el uso del motor de interpretación de Rasa, lograba identificar correctamente la intención del mensaje y generar una respuesta adecuada. Se evaluaron casos como:

- Reformulaciones simples de una pregunta (e.g. “¿cuánto es 12 entre 4?” vs. “hazme una división: 12 / 4”).
- Errores ortográficos leves (e.g. “dime una divicion” en lugar de “división”).
- Mensajes ambiguos o fuera de dominio.

En general, el sistema respondió de forma coherente y consistente en la mayoría de los casos, demostrando una adecuada capacidad de generalización dentro de los límites definidos por las historias, intents y entidades entrenadas.

En la figura 5.1 se muestra un ejemplo de interacción en el que el usuario inicia la conversación con un saludo y el bot responde adecuadamente. A continuación, al solicitar una división, el bot presenta varias opciones y el usuario selecciona la primera de ellas. Posteriormente, el usuario solicita practicar divisiones escribiendo la palabra con un error ortográfico (“praciicar” en lugar de “practicar”) y aún así, el bot es capaz de interpretar correctamente la intención, respondiendo con las opciones correspondientes.

```
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input -> Hola
¡Hola! ¿En qué te puedo ayudar hoy?
Your input -> Que es una división?
? ¿Qué tipo de división te gustaría aprender?

1: División de una cifra (/choose_one_digit_division)
¡Mira qué divertido! Hay 10 caramelos 🍬 y los dos niños los están repartiendo para que a cada uno le toque lo mismo. Es
o es dividir. 😊 Al final, cada uno recibe 5 caramelos, porque 10 dividido entre 2 es igual a 5.
Image: http://localhost:3000/1.gif
Your input -> praciicar divisiones
? ¿Como quieres practicar las divisiones?

1: División con enunciados (/choose_statment_division)
? Nivel de dificultad para tus enunciados 2: Medio (/choose_statement_difficulty{"dificultad":"medio"})
? ¿Cuántos ejercicios con enunciados quieres practicar? (Use arrow keys)
» 1: 2 enunciados (/choose_number_of_statement_exercises{"numero":"2"})
2: 4 enunciados (/choose_number_of_statement_exercises{"numero":"4"})
3: 6 enunciados (/choose_number_of_statement_exercises{"numero":"6"})
4: 10 enunciados (/choose_number_of_statement_exercises{"numero":"10"})
Type out your own message...
```

Figura 5.1: Conversación coherente con el bot.

Uso de símbolos

Estas pruebas consistieron en enviar al bot distintas formulaciones usando símbolos matemáticos. No obstante, se observó que el bot presenta mayores dificultades para interpretar correctamente los mensajes que contienen únicamente símbolos matemáticos, como “12 / 4” o “5 + 3”, sin contexto adicional. Este comportamiento se debe a las limitaciones actuales del modelo de interpretación, el cual está optimizado principalmente para entradas en lenguaje natural. A pesar de ello, el rendimiento general fue satisfactorio, y se contempla abordar esta limitación en futuras versiones del sistema. En la figura 5.2 se puede ver cómo el bot no es capaz de comprender el símbolo “+” a la hora de pedirle una suma. Tampoco es capaz de responder a una división si lleva “/”.


```
Your input -> 20 + 1
Primero genera un ejercicio de división antes de responder.
Your input -> cuanto es 20 + 1
Lo siento, no entendí lo que dijiste. ¿Puedes reformularlo?
Your input -> suma 20 más 1
La suma de 20.0 y 1.0 es 21.00.
Your input -> 20 más 1
La suma de 20.0 y 1.0 es 21.00.
Your input -> |
```

Figura 5.2: Comprobar si el bot es capaz de responder unas sumas planteadas de formas distintas.

5.2. Corrección de respuestas incorrectas

Corrección de errores

Estas pruebas se centraron en verificar que las respuestas incorrectas se almacenan correctamente durante la realización de los ejercicios, con el fin de poder ofrecer una explicación personalizada al finalizar la ronda. Una vez enviada la corrección correspondiente al usuario, el slot que almacena las divisiones incorrectas se limpia automáticamente, dejando el sistema listo para una nueva serie de ejercicios, como se puede ver en la figura 5.3

```
✓ TODOS LOS EJERCICIOS COMPLETADOS, ENVIANDO CORRECCIÓN
✓ ENVIANDO DIVISIONES INCORRECTAS A REACT: [{ 'dividendo': 45, 'divisor': 5, 'respuesta_correcta': 9, 'respuesta_usuario': 2 }, { 'dividendo': 12, 'divisor': 4, 'respuesta_correcta': 3, 'respuesta_usuario': 2 }]
✓ Enviando mensaje de corrección...
✓ Enviando imágenes de corrección...
● BORRANDO incorrect_divisions después de enviar la corrección...
```

Figura 5.3: Comprobación de almacenamiento y limpieza del slot de divisiones incorrectas

5.3. Casos de uso

Este apartado tiene como objetivo enseñar diferentes escenarios en los que los usuarios interactúan con el chatbot educativo desarrollado. Se presentan casos de uso representativos que permiten comprender cómo el sistema responde ante distintos tipos de entradas y situaciones, tanto en modo teórico como práctico, y en los diferentes niveles de dificultad.

5.3.1. Estructura de los casos de uso

Caso de uso 1: Comenzar por la teoría de divisiones

En este caso, el usuario inicia la conversación mostrando interés por repasar la teoría sobre divisiones.

Como se observa en la Figura 5.4, el usuario comienza con un saludo “hola” y el chatbot responde con un mensaje de bienvenida, indicando que puede ayudar a practicar o repasar teoría.

A continuación, el usuario pregunta “¿qué haces?”, buscando entender mejor cuál es la utilidad del chatbot. El bot responde que puede explicar cómo se hacen las divisiones y ofrecer ejercicios para practicar.

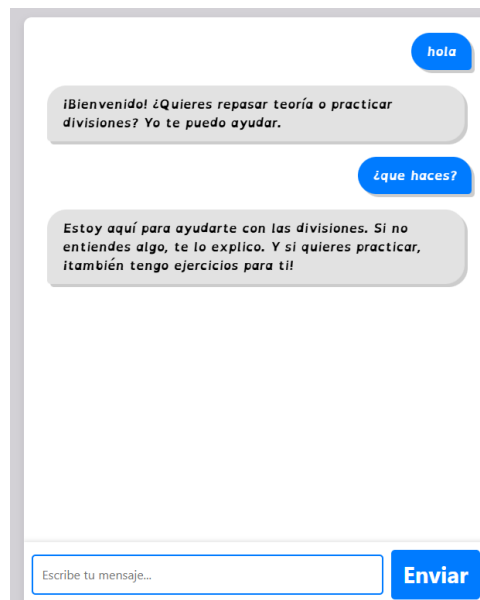


Figura 5.4: Ejemplo de interacción inicial. El usuario saluda y pregunta por la función del chatbot, quien responde explicando que puede ayudar a aprender y practicar divisiones.

Posteriormente, el usuario expresa su intención de aprender a dividir escribiendo “quiero aprender a dividir”. Como se muestra en la Figura 5.5, el chatbot responde preguntando qué tipo de división desea aprender, presentando cuatro opciones con botones visuales y accesibles:

- División de una cifra
- División de dos cifras
- División con decimales

- División con residuo

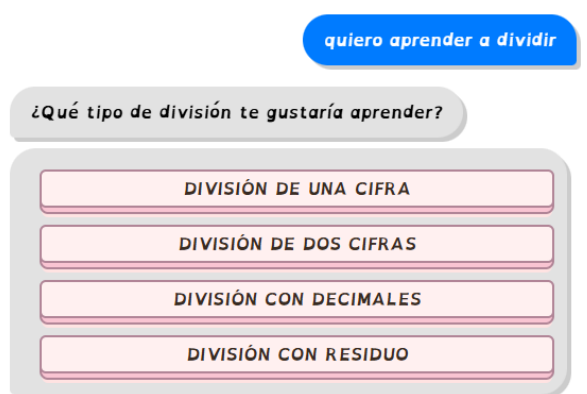


Figura 5.5: El usuario expresa su deseo de aprender a dividir. El chatbot responde con un mensaje interactivo y opciones visuales para elegir el tipo de división.

Si el usuario selecciona la opción “División de una cifra”, el chatbot responde con una explicación sencilla sobre qué significa dividir. Como se muestra en la Figura 5.6, el bot presenta una situación cotidiana: dos niños reparten 10 caramelos de forma equitativa, de modo que cada uno recibe 5. Esta explicación se acompaña de una animación (GIF) donde los caramelos se van repartiendo visualmente entre los niños.

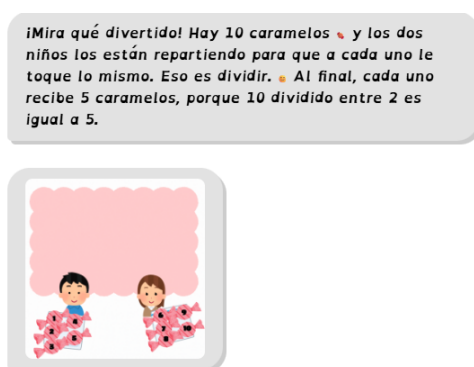


Figura 5.6: Explicación visual y animada de una división de una cifra. El chatbot muestra cómo se reparten 10 caramelos entre dos niños, reforzando el concepto de división como reparto igualitario.

En el caso de que el usuario seleccione la opción “División con decimales”, el chatbot ofrece una explicación paso a paso . Además, se proporciona una imagen visual que muestra cómo se

resuelve una división con decimales, incluyendo la colocación del punto decimal y los pasos intermedios.

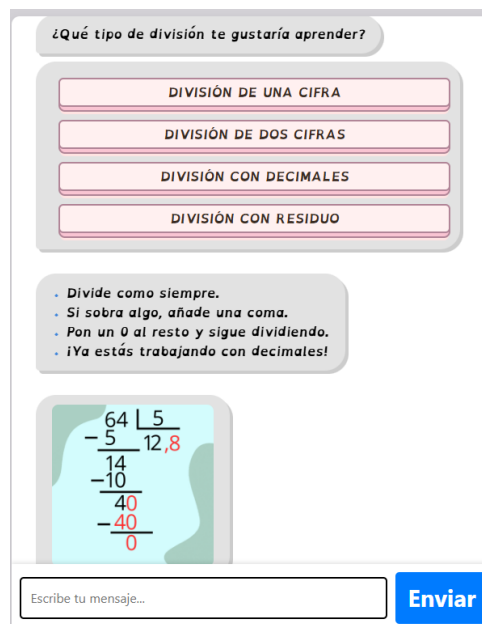


Figura 5.7: Explicación visual y textual sobre cómo resolver divisiones con decimales. El chatbot guía al usuario con instrucciones sencillas y una representación gráfica del procedimiento.

De forma similar, si el usuario selecciona “División con residuo”, el bot explica de manera contextual qué significa que sobre algo al dividir. Utiliza un ejemplo con caramelos para facilitar la comprensión, acompañado de una imagen ilustrativa amigable para los niños.

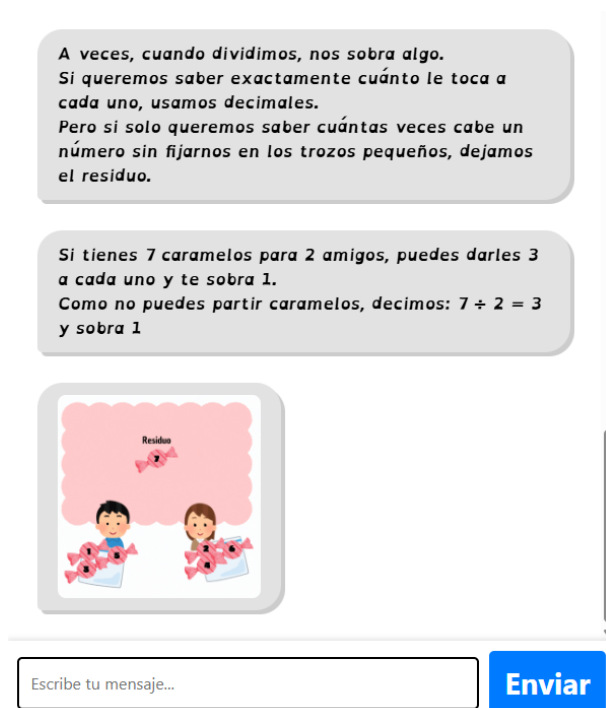


Figura 5.8: Explicación visual y textual sobre cómo resolver divisiones con decimales. El chatbot guía al usuario con instrucciones sencillas y una representación gráfica del procedimiento.

Por último, si el usuario selecciona la opción “División de dos cifras”, el chatbot presenta una breve explicación. En esta, se aclara que el número por el que se divide (el divisor) tiene dos cifras y se ejemplifica con la operación $144 \div 12$. Tal como se muestra en la Figura 5.9, el bot explica cómo entender esta operación.

En las divisiones de dos cifras, el divisor tiene dos números. Ejemplo: $144 \div 12 = 12$. Esto quiere decir que vamos a ver cuántas veces 12 cabe dentro de 144. Si hacemos la cuenta paso a paso, descubrimos que $12 \times 12 = 144$, así que $144 \div 12 = 12$.

Figura 5.9: Explicación textual sobre cómo resolver divisiones con un divisor de dos cifras. El chatbot utiliza un ejemplo numérico simple para facilitar la comprensión.

Caso de uso 2: Practicar divisiones

En este caso, el usuario desea practicar divisiones. Como se observa en la Figura 5.10, el usuario inicia la conversación con un saludo y expresa su intención: “quiero practicar divisiones”.

El chatbot responde de forma clara y estructurada, guiando al usuario a través de una serie

de opciones. Primero, le pregunta cómo quiere practicar: con ejercicios sueltos o mediante enunciados. A continuación, ofrece una selección de niveles de dificultad: fácil, medio, difícil o experto.

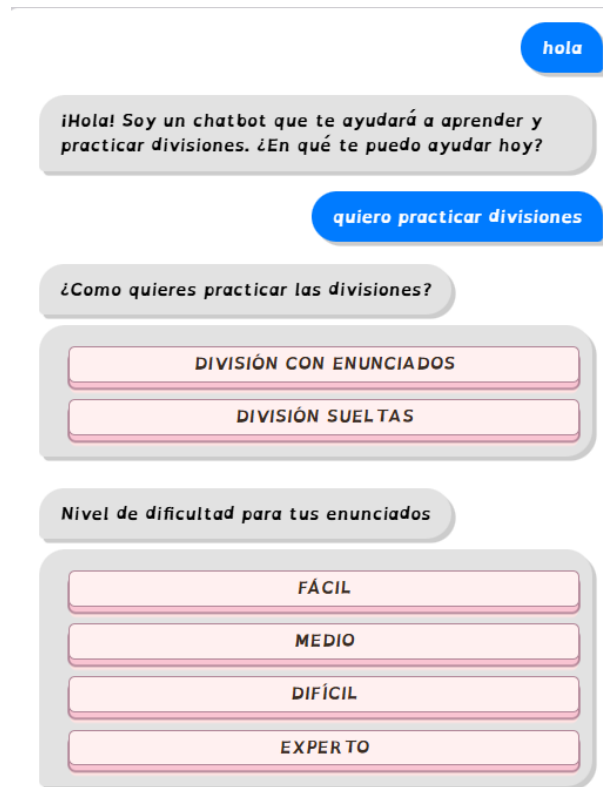


Figura 5.10: Interacción en la que el usuario selecciona cómo desea practicar divisiones. El chatbot ofrece opciones visuales para elegir el tipo de ejercicio y el nivel de dificultad.

Tras seleccionar el tipo de ejercicio y el nivel de dificultad, que fue fácil, el chatbot pregunta al usuario cuántos enunciados quiere practicar, ofreciendo varias opciones.

Como se muestra en la Figura 5.11, el bot presenta una serie de ejercicios con enunciados adaptados al nivel elegido. El usuario responde con un número y el chatbot ofrece retroalimentación inmediata: si la respuesta es correcta, refuerza positivamente con frases como "¡Correcto! Buen trabajo." Si la respuesta es incorrecta, el chatbot proporciona la solución correcta como se puede ver en la figura 5.12

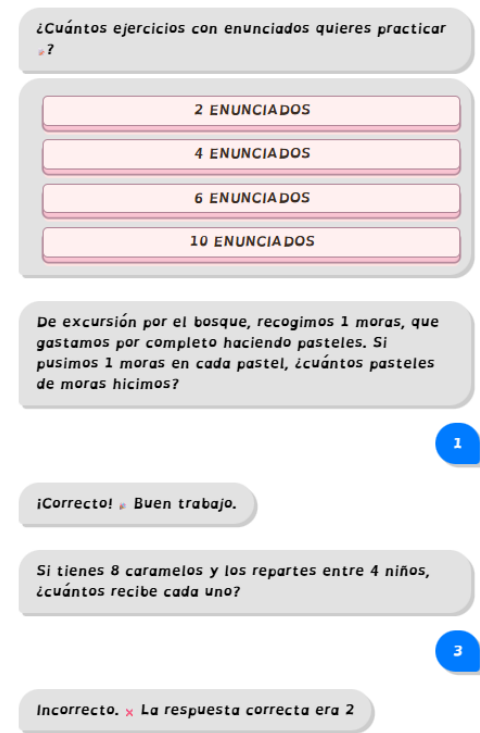


Figura 5.11: El usuario elige la cantidad de ejercicios a practicar. El chatbot muestra enunciados y proporciona feedback inmediato según la respuesta.

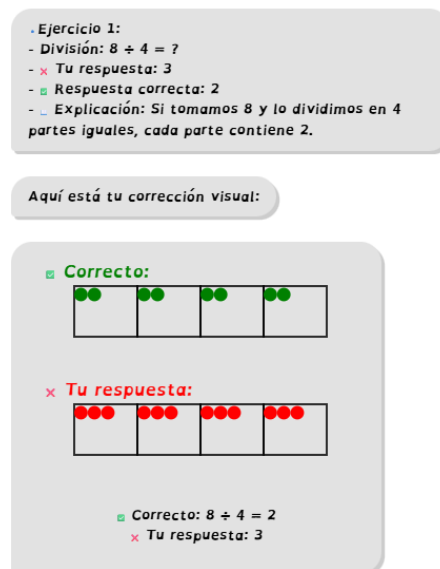


Figura 5.12: Corrección visual y explicada que proporciona el chatbot.

Capítulo 6

Conclusiones

6.1. Consecución de objetivos

Este capítulo se centra en la revisión y valoración de los propósitos generales y particulares formulados al comienzo de este Trabajo de Fin de Grado. Aquí reflexionamos sobre los avances logrados, identificamos los retos enfrentados y revisamos las estrategias utilizadas para superarlos, además de analizar aquellos aspectos que no se han logrado resolver.

A lo largo del desarrollo del proyecto se han cumplido de manera satisfactoria los objetivos principales planteados. Se ha logrado diseñar e integrar un chatbot educativo basado en tecnologías de procesamiento del lenguaje natural, capaz de guiar al usuario en la resolución de divisiones. Asimismo, se ha implementado una interfaz visual atractiva utilizando herramientas modernas de desarrollo front-end, pensada específicamente para un público infantil.

El sistema permite ajustar la dificultad de los ejercicios, adaptándose progresivamente al ritmo de aprendizaje del usuario e incluyendo tanto divisiones exactas como no exactas. Además, la aplicación funciona completamente en un entorno digital accesible, gratuito y sin necesidad de registro ni instalación, cumpliendo con los requisitos de accesibilidad y simplicidad de uso.

No obstante, uno de los desafíos más significativos que, lamentablemente, no se logró alcanzar fue la implementación del reconocimiento de voz, cuyo objetivo era permitir que el usuario pudiera interactuar con el chatbot sin necesidad de escribir. Esta funcionalidad tuvo que ser descartada debido a las limitaciones de tiempo, ya que habría requerido una investigación exhaustiva de tecnologías compatibles y el estudio de su integración, lo cual implicaba una curva de aprendizaje adicional considerable. Se considera, sin embargo, una posible línea de

mejora para futuros desarrollos del sistema.

En resumen, al dar por finalizado este Trabajo de Fin de Grado, se puede afirmar que se han cumplido la gran mayoría de las metas propuestas inicialmente. Los obstáculos surgidos durante el desarrollo se transformaron en valiosas oportunidades de aprendizaje, permitiendo no solo alcanzar los objetivos planteados, sino también ampliar el conocimiento técnico y pedagógico sobre el diseño de herramientas educativas accesibles.

6.2. Aplicación de lo aprendido

Durante el Grado he cursado diversas asignaturas relacionadas con la programación, lo cual me ha permitido desarrollar una base sólida para afrontar este Trabajo de Fin de Grado. Desde los primeros cursos, las asignaturas obligatorias de programación han contribuido a que adquiriera habilidades en lógica computacional, estructuras de control y diseño de algoritmos, todas ellas fundamentales para estructurar el flujo conversacional y la lógica del chatbot.

Durante la etapa final de la carrera, elegí asignaturas optativas más especializadas, entre las que destacan **Construcción de Servicios y Aplicaciones Audiovisuales en Internet** y **Laboratorio de Tecnologías Audiovisuales en la Web**. La primera me permitió aprender a construir interfaces modernas y trabajar con HTML5, JavaScript y CSS. Además, me ayudó a crear experiencias interactivas, conocimientos que he aplicado directamente en el desarrollo del front-end del proyecto. La segunda se centraba en el diseño de páginas web completas y la integración con bases de datos, lo que me permitió comprender la importancia de separar la lógica del cliente y del servidor, así como la comunicación entre ambos componentes.

Gracias a esta formación, he podido desarrollar un sistema completo donde el front-end se comunica con el back-end (Rasa) mediante API REST, se gestionan interacciones dinámicas con el usuario y se implementan visualizaciones personalizadas para reforzar el aprendizaje.

6.3. Lecciones aprendidas

Aparte de los conocimientos proporcionados en la carrera, se han obtenido nuevas competencias y se ha profundizado en otras para superar los desafíos encontrados durante el desarrollo del proyecto.

1. Comprensión del funcionamiento interno de RASA. Se ha adquirido una visión general del framework, incluyendo su arquitectura y componentes principales.
2. Aprendizaje práctico de React. Se ha trabajado con esta biblioteca para construir una interfaz web interactiva y atractiva, mejorando habilidades en desarrollo front-end y en la integración con otros sistemas como RASA.
3. Comprensión del funcionamiento de intro.js. Se ha aprendido a integrar esta biblioteca para implementar tours guiados que mejoran la experiencia de usuario en la aplicación.
4. Utilización de LaTeX para la elaboración de la memoria. Se ha ganado experiencia en el uso de esta herramienta, ampliamente utilizada en el ámbito académico para la creación de documentos técnicos y estructurados.

6.4. Trabajos futuros

Este apartado recoge posibles mejoras o extensiones que podrían incorporarse en el futuro.

- **Incorporación de minijuegos:** Desarrollar juegos interactivos que otorguen puntos al usuario cada vez que responda correctamente una división, fomentando así la motivación y el aprendizaje divertido.
- **Sistema de recompensas y ranking:** Implementar un sistema de puntuación acumulativa y clasificación entre usuarios permitiría introducir dinámicas de competencia saludable. Esto podría incentivarse aún más con logros, medallas o niveles.
- **Personalización por parte de los padres o docentes:** Incorporar un módulo donde familiares o educadores pudieran introducir manualmente enunciados personalizados, adaptados a los intereses, necesidades o dificultades específicas del niño o la niña.
- **Ampliación a otras materias:** Implementar ejercicios de otras materias, como lengua o ciencias, además de operaciones matemáticas básicas.
- **Historial de respuestas y análisis del progreso:** Guardar las respuestas del usuario permitiría ofrecer estadísticas de mejora, detectar patrones de error y generar recomendaciones personalizadas.

Apéndice A

Manual de usuario

A continuación se detallará una guía para que el usuario pueda hacer uso del chatbot. En primer lugar, se explica el proceso de instalación del mismo y, a continuación, se describe una guía para el funcionamiento. El proceso está descrito para el sistema operativo Windows; es funcional también en Linux.

A.1. Instalación

- **Instalación Python:** Se recomienda instalar una versión de Python 3.8 o superior, la que se usó para este proyecto fue 3.9.12
- **Instalación Rasa:** La versión de RASA usada para el proyecto fue la 3.6.20
pip install rasa
- **Instalación de bibliotecas:**
 - **Bootstrap:** *npm install bootstrap*
 - **Intro.js:** *npm install intro.js*
 - **spaCy y el modelo es_core_news_md:** *pip install spacy*
python -m spacy download es_core_news_md
- **Instalación de Node.js y npm:** Necesarios para ejecutar el front-end en React y gestionar paquetes como Bootstrap o Intro.js. Descarga desde: <https://nodejs.org>

- **Clonación de repositorio:** Se puede clonar el repositorio en el perfil de GITHUB: “<https://github.com/Cecilia-Kholin/TFG-chatbot.git>” para obtener el chatbot.

A.2. Guía usuario

Antes de utilizar el bot, es necesario entrenarlo, ya que los archivos generados durante este proceso (como los modelos) pueden tener un tamaño considerable y no se incluyen en el repositorio. Para entrenarlo, se debe ejecutar el siguiente comando desde la raíz del proyecto:

```
rasa train
```

Una vez entrenado, para que el chatbot funcione correctamente es necesario desplegar los siguientes tres componentes:

```
npm start  
rasa run --enable-api --cors "*"   
rasa run actions
```

Este comando realiza lo siguiente:

- `--enable-api`: Activa la API REST de RASA, lo que permite que el front-end (por ejemplo, una aplicación React) envíe y reciba mensajes del bot mediante peticiones HTTP.
- `--cors "*"`: Permite solicitudes CORS (Cross-Origin Resource Sharing) desde cualquier origen. Esto es necesario cuando el front-end y el back-end (RASA) están en servidores o puertos distintos, como ocurre durante el desarrollo.

Cuando todo esté desplegado, se podrá probar el chatbot.

Bibliografía

- [1] UNESCO. Educación: Datos y cifras. <https://www.unesco.org/es/articles/la-unesco-advierte-que-de-no-tomar-medidas-urgentes-de-accion> 2023. Último acceso: 19 de mayo de 2025.
- [2] Unión Internacional de Telecomunicaciones (UIT). Measuring digital development: Facts and figures 2023. <https://www.itu.int/en/ITU-D/Statistics/Pages/stat/default.aspx>, 2023. International Telecommunication Union.
- [3] UNICEF. Education: Overview. <https://data.unicef.org/topic/education/overview/>, 2024. Fondo de las Naciones Unidas para la Infancia.
- [4] Álvaro Castillo Cabero. Rasa framework-análisis e implementación de un chatbot. B.S. thesis, Universitat Politècnica de Catalunya, 2020.
- [5] Rasa Technologies. Rasa documentation, 2025. Último acceso: abril de 2025.
- [6] Javier Eguíluz Pérez. introduccion a javascript, 2019.
- [7] Santiago Aguirre. *HTML5 Avanzado 1: Formularios Avanzados-Contenido Responsive-SEO*. RedUsers, 2021.
- [8] Raúl Tabarés. Html5 and the evolution of html; tracing the origins of digital platforms. *Technology in Society*, 65:101529, 2021.
- [9] Sufyan bin Uzayr. *Mastering CSS: A Beginner's Guide*. CRC Press, 2023.
- [10] Intro.js. Intro.js documentation, 2025. Accedido el 27 de abril de 2025.

- [11] Kornel Maciej Chromiński, L'ubomír Benko, Zenón José Hernández-Figueroa, José Daniel González-Domínguez, Juan Carlos Rodríguez-Del-Pino, and Jan Přichystal. *Python fundamentals*, 2021.
- [12] json. *json documentation*, 2025. Accedido el 27 de abril de 2025.
- [13] Explosion. *spacy · industrial-strength natural language processing in python*.
- [14] Julen Astigarraga and Verónica Cruz-Alonso. ¿ se puede entender cómo funcionan git y github! *ecosistemas*, 31(1):2332–2332, 2022.
- [15] Microsoft. *Visual studio code*. <https://code.visualstudio.com/>, 2024.
- [16] Helmut Kopka and Patrick W Daly. *Guide to LATEX*. Pearson Education, 2003.
- [17] Abelardo Gonzalez. *Open dyslexic*. <https://opendyslexic.org>, 2011. Tipografía de código abierto diseñada para personas con dislexia.